NUMPY PRACTICE SET

Questions & Answers

55 exercises from beginner to real-world, plus 6 slicing drills. Every question is followed directly by its answer, output and a short note on the better way to do it.

Level	Questions	Focus
Level 1 - Beginner	1-12	Array creation, shape and data types
Level 2 - Basic Operations	13-24	Indexing, slicing and arithmetic
Level 3 - Intermediate	25-36	Filtering, aggregation and reshaping
Level 4 - Advanced	37-48	Broadcasting, matrices and vectorization
Level 5 - Real-World Challenges	49-55	Data analysis scenarios
Day 1 - Slicing Drills	Extra practice	Indexing and slicing with steps

All code assumes `import numpy as np`. Arrays that were missing from the original chat were rebuilt, and every answer was run to produce the output shown.

Level 1 - Beginner

Questions 1-12 | Array creation, shape and data types

Q1 Create a NumPy array

QUESTION

Create a 1D array containing 10, 20, 30, 40, 50. Store it in `arr` and print it.

ANSWER

PYTHON

```python
arr = np.array([10, 20, 30, 40, 50])
print(arr)
```

OUTPUT

```
[10 20 30 40 50]
```

ALTERNATIVE

```python
arr = np.arange(10, 51, 10)
print(arr)
```

Use `np.array()` when you know the exact values. Use `np.arange()` when the values follow a regular step (remember the stop value `51` is excluded).

Q2 Create a 2D array

QUESTION

Create a 3 x 3 array containing the numbers 1 to 9. Store it in `arr` and print it.

ANSWER

PYTHON

```python
arr = np.arange(1, 10).reshape(3, 3)
print(arr)
```

OUTPUT

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

ALTERNATIVE

```python
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

Nested lists are explicit. `arange` + `reshape` is shorter for a continuous sequence.

Q3 Create an array of zeros

QUESTION

Create a 1D array of 8 zeros and print it.

ANSWER

PYTHON

```python
print(np.zeros(8))
```

OUTPUT

```
[0. 0. 0. 0. 0. 0. 0. 0.]
```

ALTERNATIVE

```python
print(np.zeros(8, dtype=int))
```

`np.zeros()` gives floats by default. Add `dtype=int` if you need integer zeros.

Q4 Create an array of ones

QUESTION

Create a 3 x 4 array where every element is 1.

ANSWER

PYTHON

```python
print(np.ones((3, 4)))
```

OUTPUT

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

ALTERNATIVE

```python
print(np.ones((3, 4), dtype=int))
```

For 2D arrays the shape goes in as a **tuple**: `(3, 4)`, not `3, 4`.

Q5 Generate a sequence

QUESTION

Create an array of integers from 1 to 20, excluding 20.

ANSWER

PYTHON

```python
print(np.arange(1, 20))
```

OUTPUT

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
```

`np.arange(start, stop, step)` never includes `stop`. The default step is 1.

Q6 Evenly spaced values

QUESTION

Create 5 evenly spaced numbers between 0 and 100, including both endpoints.

ANSWER

PYTHON

```python
print(np.linspace(0, 100, 5))
```

OUTPUT

```
[  0.  25.  50.  75. 100.]
```

`np.linspace(start, stop, num)` **includes** the endpoint. Common mistake: writing `101` as the stop.

Q7 Check the shape

QUESTION

Create a 4 x 5 array containing 1 to 20. Print its shape, number of dimensions and total number of elements.

ANSWER

PYTHON

```python
arr = np.arange(1, 21).reshape(4, 5)
print(arr.shape)
print(arr.ndim)
print(arr.size)
```

OUTPUT

```
(4, 5)
2
20
```

`reshape` needs rows x columns = total elements. `reshape(4, 3)` on 20 elements raises a `ValueError`.

Q8 Check the data type

QUESTION

Create an integer array `[10, 20, 30]` and a float array `[1.5, 2.5, 3.5]`. Print the data type of each.

ANSWER

PYTHON

```python
a = np.array([10, 20, 30])
b = np.array([1.5, 2.5, 3.5])
print(a.dtype)
print(b.dtype)
```

OUTPUT

```
int64
float64
```

ALTERNATIVE

```python
a = np.array([10, 20, 30], dtype=np.int32)
print(a.dtype)
```

`np.array()` takes **one list**, not separate numbers. `dtype` is an attribute: `a.dtype`, not `dtype(a)`. The exact integer type can vary by system.

Q9 Convert data types

QUESTION

Create `[1, 2, 3, 4, 5]` as a floating-point array and print the array and its dtype.

ANSWER

PYTHON

```python
arr = np.array([1, 2, 3, 4, 5], dtype=float)
print(arr)
print(arr.dtype)
```

OUTPUT

```
[1. 2. 3. 4. 5.]
float64
```

ALTERNATIVE

```python
arr = np.array([1, 2, 3, 4, 5]).astype(float)
print(arr)
print(arr.dtype)
```

`dtype=` sets the type at creation. `.astype()` converts an existing array.

Q10 Identity matrix

QUESTION

Create a 4 x 4 identity matrix and print it.

ANSWER

PYTHON

```python
print(np.eye(4))
```

OUTPUT

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

`np.eye(4, 4)` also works, but `np.eye(4)` is already 4 x 4. Identity matrix: 1s on the diagonal, 0s elsewhere.

Q11 Random integers

QUESTION

Generate 10 random integers between 1 and 100, inclusive.

ANSWER

PYTHON

```python
rng = np.random.default_rng(42)   # seed makes the result repeatable
arr = rng.integers(1, 101, 10)
print(arr)
```

OUTPUT

```
[ 9 78 66 44 44 86  9 70 21 10]
```

ALTERNATIVE

```python
arr = np.random.randint(1, 101, 10)
print(arr)
```

The upper bound (`101`) is **excluded**, so values run from 1 to 100. `default_rng()` is the modern style; `np.random.randint` is the older one you will see everywhere. Without a seed the numbers change on every run.

Q12 Reshape an array

QUESTION

Create the numbers 1 to 12 and reshape them into 3 rows and 4 columns.

ANSWER

PYTHON

```python
arr = np.arange(1, 13)
res_arr = arr.reshape(3, 4)
print(res_arr)
```

OUTPUT

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

ALTERNATIVE

```python
print(np.arange(1, 13).reshape(3, 4))
```

Reshaping never changes the number of elements: 3 x 4 = 12.

Level 2 - Basic Operations

Questions 13-24 | Indexing, slicing and arithmetic

Q13 Access elements

QUESTION

Print the first, fourth and last elements of `arr`.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
```

ANSWER

PYTHON

```
print(arr[0])
print(arr[3])
print(arr[-1])
```

OUTPUT

```
10
40
80
```

ALTERNATIVE

```
print(arr[[0, 3, -1]])
```

Indexing starts at 0, so the fourth element is `arr[3]`. Negative indexing counts from the end: `-1` is the last element.

Q14 Negative indexing

QUESTION

Print the last three elements of `arr`.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
```

ANSWER

PYTHON

```
print(arr[-3:])
```

OUTPUT

```
[60 70 80]
```

`arr[-3:]` means: start at the third element from the end and go to the end.

Q15 Slicing

QUESTION

Extract `[20, 30, 40, 50]` from `arr` using slicing.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
```

ANSWER

PYTHON

```
print(arr[1:5])
```

OUTPUT

```
[20 30 40 50]
```

In a slice the start index is included and the stop index is excluded.

Q16 Reverse an array

QUESTION

Reverse `arr` without a Python loop.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
```

ANSWER

PYTHON

```
print(arr[::-1])
```

OUTPUT

```
[80 70 60 50 40 30 20 10]
```

ALTERNATIVE

```
print(np.flip(arr))
```

`[::-1]` uses a step of -1. Don't confuse it with `[:-1]`, which only **drops the last element**. `np.flip()` also works along a chosen axis for 2D arrays.

Q17 Modify elements

QUESTION

Change the third element to 300 and the last element to 800, then print the array.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
```

ANSWER

PYTHON

```python
arr[2] = 300
arr[-1] = 800
print(arr)
```

OUTPUT

```
[ 10  20 300  40  50  60  70 800]
```

The third element is index **2**, not 4. `arr[-6] = 300` would also work on this 8-element array, but `arr[2]` is clearer.

Q18 Element-wise addition

QUESTION

Add 10 to every element and print the result.

GIVEN

```python
arr = np.array([10, 20, 30, 40, 50])
```

ANSWER

PYTHON

```python
print(arr + 10)
```

OUTPUT

```
[20 30 40 50 60]
```

ALTERNATIVE

```python
print(np.add(arr, 10))
```

Don't forget to `print()` (or display) the result. `arr + 10` creates a new array and leaves `arr` unchanged.

Q19 Element-wise multiplication

QUESTION

Multiply every element by 3 and print the result.

GIVEN

```python
arr = np.array([10, 20, 30, 40, 50])
```

ANSWER

PYTHON

```python
print(arr * 3)
```

OUTPUT

```
[ 30  60  90 120 150]
```

ALTERNATIVE

```
print(np.multiply(arr, 3))
```

This is vectorization: arithmetic applies to the whole array with no loop.

Q20 Arithmetic between arrays

QUESTION

Add the two arrays element by element.

GIVEN

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
```

ANSWER

PYTHON

```
print(a + b)
```

OUTPUT

```
[11 22 33 44]
```

ALTERNATIVE

```
print(np.add(a, b))
```

Both arrays have shape `(4,)`, so corresponding elements are added.

Q21 Square every element

QUESTION

Calculate the square of each value in `arr`.

GIVEN

```
arr = np.array([1, 2, 3, 4, 5])
```

ANSWER

PYTHON

```
print(arr ** 2)
```

OUTPUT

```
[ 1  4  9 16 25]
```

ALTERNATIVE

```
print(arr * arr)
print(np.square(arr))
```

All three give the same result. `arr ** 2` reads most like the maths.

Q22 Min, max, sum and mean

QUESTION

Find the minimum, maximum, sum and mean of `arr`.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50])
```

ANSWER

PYTHON

```
print(np.min(arr))
print(np.max(arr))
print(np.sum(arr))
print(np.mean(arr))
```

OUTPUT

```
10
50
150
30.0
```

ALTERNATIVE

```
print(arr.min(), arr.max(), arr.sum(), arr.mean())
```

Methods (`arr.sum()`) and functions (`np.sum(arr)`) are equivalent. Mean = sum / count = 150 / 5 = 30.

Q23 Sort an array

QUESTION

Sort `arr` in ascending and in descending order.

GIVEN

```
arr = np.array([45, 12, 78, 34, 23])
```

ANSWER

PYTHON

```
print(np.sort(arr))
print(np.sort(arr)[::-1])
```

OUTPUT

```
[12 23 34 45 78]
[78 45 34 23 12]
```

ALTERNATIVE

```
print(-np.sort(-arr))
```

Descending order = sort, then reverse with `[::-1]`. Don't use `[:-1]` : that drops the last element instead of reversing.

Q24 Join arrays

QUESTION

Concatenate the two arrays into one and print it.

GIVEN

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
```

ANSWER

PYTHON

```
print(np.concatenate((a, b)))
```

OUTPUT

```
[1 2 3 4 5 6]
```

ALTERNATIVE

```
print(np.append(a, b))
```

`concatenate` is a NumPy **function** (`np.concatenate`), not an array method. It is preferred because it joins several arrays along any axis.

Level 3 - Intermediate

Questions 25-36 | Filtering, aggregation and reshaping

Q25 Filter even numbers

QUESTION

From the integers 1 to 50, extract and print only the even numbers.

ANSWER

PYTHON

```python
arr = np.arange(1, 51)
print(arr[arr % 2 == 0])
```

OUTPUT

```
[ 2  4  6  8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48
 50]
```

ALTERNATIVE

```python
arr = np.arange(1, 51)
print(arr[np.where(arr % 2 == 0)])
print(np.arange(2, 51, 2))
```

Boolean masking is the shortest and most readable way to filter. `np.where(cond)` returns **indices**, which you then use to index the array.

Q26 Filter by condition

QUESTION

Select all values greater than 30.

GIVEN

```python
arr = np.array([12, 45, 7, 89, 34, 56, 2])
```

ANSWER

PYTHON

```python
print(arr[arr > 30])
```

OUTPUT

```
[45 89 34 56]
```

ALTERNATIVE

```python
print(arr[np.where(arr > 30)])
```

`arr > 30` builds a True/False array. Indexing with it keeps only the True positions.

Q27 Replace values conditionally

QUESTION

Replace every negative number with zero.

GIVEN

```
arr = np.array([-5, 10, -2, 8, -9, 4])
```

ANSWER

PYTHON

```
print(np.where(arr < 0, 0, arr))
```

OUTPUT

```
[ 0 10  0  8  0  4]
```

ALTERNATIVE

```
arr[arr < 0] = 0      # modifies arr in place
print(arr)
```

`np.where(condition, value_if_true, value_if_false)`. There is no `? :` ternary in Python. `np.where` returns a **new** array; Boolean assignment changes the original.

Q28 Conditional labels with np.where

QUESTION

Return "Pass" if the value is at least 25, otherwise "Fail".

GIVEN

```
arr = np.array([10, 25, 30, 15, 40])
```

ANSWER

PYTHON

```
print(np.where(arr >= 25, "Pass", "Fail"))
```

OUTPUT

```
['Fail' 'Pass' 'Pass' 'Fail' 'Pass']
```

Check your brackets: a missing closing `)` is the most common error here.

Q29 Count matching elements

QUESTION

Count how many elements are equal to 2.

GIVEN

```
arr = np.array([2, 5, 8, 2, 9, 2, 4, 8])
```

ANSWER

PYTHON

```python
print(np.count_nonzero(arr == 2))
```

OUTPUT

```
3
```

ALTERNATIVE

```python
print(np.sum(arr == 2))
print(len(np.where(arr == 2)[0]))
```

`np.where(arr == 2)` returns the **indices** of matches, so summing it is wrong. `True` counts as 1, so `np.sum(arr == 2)` works, but `np.count_nonzero` states the intent best.

Q30 Find unique values

QUESTION

Return all unique values in the array.

GIVEN

```python
arr = np.array([1, 2, 2, 3, 4, 4, 4, 5])
```

ANSWER

PYTHON

```python
print(np.unique(arr))
```

OUTPUT

```
[1 2 3 4 5]
```

`np.unique` removes duplicates and returns the values sorted. Python's `set()` would return a set with no guaranteed order.

Q31 Count occurrences

QUESTION

Find the frequency of each unique value.

GIVEN

```python
arr = np.array([1, 2, 2, 3, 4, 4, 4, 5])
```

ANSWER

PYTHON

```python
values, counts = np.unique(arr, return_counts=True)
print(values)
print(counts)
```

OUTPUT

```
[1 2 3 4 5]
[1 2 1 3 1]
```

The two arrays line up position by position: value 4 appears 3 times.

Q32 Row sums

QUESTION

Calculate the sum of each row.

GIVEN

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

ANSWER

PYTHON

```
print(np.sum(arr, axis=1))
```

OUTPUT

```
[ 6 15 24]
```

ALTERNATIVE

```
print(arr.sum(axis=1))
```

`axis=1` collapses the columns, giving one result per row: 1+2+3 = 6, 4+5+6 = 15, 7+8+9 = 24.

Q33 Column averages

QUESTION

Find the mean of each column.

GIVEN

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

ANSWER

PYTHON

```
print(np.mean(arr, axis=0))
```

OUTPUT

```
[4. 5. 6.]
```

ALTERNATIVE

```
print(arr.mean(axis=0))
```

`axis=0` collapses the rows, giving one result per column. Check you used `mean`, not `sum`.

Q34 Flatten a matrix

QUESTION

Convert a 3 x 3 matrix into a 1D array.

GIVEN

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

ANSWER

PYTHON

```
print(arr.flatten())
```

OUTPUT

```
[1 2 3 4 5 6 7 8 9]
```

ALTERNATIVE

```
print(arr.ravel())
```

`flatten()` is an array method (there is no `np.flatten`). It always returns a copy; `ravel()` returns a view when possible.

Q35 Transpose a matrix

QUESTION

Change a matrix of shape (2, 3) into one of shape (3, 2).

GIVEN

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
```

ANSWER

PYTHON

```
print(arr.T)
```

OUTPUT

```
[[1 4]
 [2 5]
 [3 6]]
```

ALTERNATIVE

```
print(np.transpose(arr))
```

Transposing swaps rows and columns.

Q36 Index of the maximum

QUESTION

Find the position of the largest element.

GIVEN

```
arr = np.array([12, 88, 34, 99, 56])
```

ANSWER

PYTHON

```
print(np.argmax(arr))
```

OUTPUT

```
3
```

ALTERNATIVE

```
print(np.where(arr == np.max(arr)))
```

`np.max` gives the **value** (99). `np.argmax` gives its **index** (3). `np.where(arr == arr.max())` returns every index when there are ties.

Level 4 - Advanced

Questions 37-48 | Broadcasting, matrices and vectorization

Q37 Broadcasting with a scalar

QUESTION

Add 10 to every element of the matrix.

GIVEN

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

ANSWER

PYTHON

```
print(arr + 10)
```

OUTPUT

```
[[11 12 13]
 [14 15 16]
 [17 18 19]]
```

ALTERNATIVE

```
print(np.add(arr, 10))
```

NumPy stretches the single number 10 across the whole matrix. Adding a different value to each column is Question 38.

Q38 Broadcasting with a 1D array

QUESTION

Add `b` to every row of `arr`.

GIVEN

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
b = np.array([100, 200, 300])
```

ANSWER

PYTHON

```
print(arr + b)
```

OUTPUT

```
[[101 202 303]
 [104 205 306]
 [107 208 309]]
```

ALTERNATIVE

```
print(np.add(arr, b))
```

Shapes: `arr` is (3, 3) and `b` is (3,). The 3 elements of `b` match the 3 columns, so `b` is applied to each row. No `axis` argument is needed.

Q39 Min-max normalization

QUESTION

Scale the array so every value is between 0 and 1.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50])
```

ANSWER

PYTHON

```
norm = (arr - arr.min()) / (arr.max() - arr.min())
print(norm)
```

OUTPUT

```
[0.   0.25 0.5  0.75 1.  ]
```

ALTERNATIVE

```
norm = (arr - arr.min()) / np.ptp(arr)
print(norm)
```

Formula: (x - min) / (max - min). `np.ptp(arr)` is max minus min.

Q40 Standardize with z-scores

QUESTION

Calculate the z-score of each element using the population standard deviation.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50])
```

ANSWER

PYTHON

```
z = (arr - arr.mean()) / arr.std()
print(z)
```

OUTPUT

```
[-1.41421356 -0.70710678  0.          0.70710678  1.41421356]
```

ALTERNATIVE

```
z = (arr - np.mean(arr)) / np.std(arr)
print(z)
```

z = (x - mean) / std. `np.std` uses the population formula (`ddof=0`) by default. Pass `ddof=1` for the sample version.

Q41 Matrix multiplication

QUESTION

Multiply a 2 x 3 matrix by a 3 x 2 matrix.

GIVEN

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([[7, 8],
              [9, 10],
              [11, 12]])
```

ANSWER

PYTHON

```
print(a @ b)
```

OUTPUT

```
[[ 58  64]
 [139 154]]
```

ALTERNATIVE

```
print(a.dot(b))
print(np.matmul(a, b))
```

`@` is the clearest operator for matrix multiplication. (2, 3) x (3, 2) gives (2, 2). `a * b` would be element-wise and would fail here.

Q42 Diagonal and its sum

QUESTION

Extract the main diagonal of a 4 x 4 matrix and calculate its sum.

GIVEN

```
arr = np.arange(1, 17).reshape(4, 4)
```

ANSWER

PYTHON

```
diag = np.diagonal(arr)
print(diag)
print(diag.sum())
```

OUTPUT

```
[ 1  6 11 16]
34
```

ALTERNATIVE

```
print(np.trace(arr))
```

`np.trace` returns only the sum of the diagonal, not the diagonal itself.

Q43 Find duplicate values

QUESTION

Find all values that appear more than once, without a loop.

GIVEN

```
arr = np.array([1, 2, 2, 3, 4, 2, 3, 5, 6])
```

ANSWER

PYTHON

```
values, counts = np.unique(arr, return_counts=True)
print(values[counts > 1])
```

OUTPUT

```
[2 3]
```

Count each unique value, build the mask `counts > 1`, then use it to pick the matching values.

Q44 Top 3 values

QUESTION

Find the three largest values without a Python loop.

GIVEN

```
arr = np.array([45, 99, 12, 85, 33, 73, 60])
```

ANSWER

PYTHON

```
top3 = np.sort(arr)[-3:][::-1]
print(top3)
```

OUTPUT

```
[99 85 73]
```

ALTERNATIVE

```
top3 = np.partition(arr, -3)[-3:]
print(np.sort(top3)[::-1])
```

Sort ascending, take the last three, then reverse them. `np.partition` avoids sorting the whole array, which helps on large data. A comparison like `arr < x` gives a Boolean mask, not the numbers.

Q45 Replace outliers

QUESTION

Replace values greater than mean + 2 standard deviations with the mean.

GIVEN

```
arr = np.array([10, 12, 11, 13, 12, 14, 100])
```

ANSWER

PYTHON

```
mean = arr.mean()
std = arr.std()
result = np.where(arr > mean + 2 * std, mean, arr)
print(result)
```

OUTPUT

```
[10.         12.         11.         13.         12.         14.
 24.57142857]
```

ALTERNATIVE

```
arr = arr.astype(float)
arr[arr > arr.mean() + 2 * arr.std()] = arr.mean()
print(arr)
```

Here the mean is about 24.57 and the threshold about 86.2, so only `100` is replaced. `np.where` returns a new array; Boolean assignment edits the original.

Q46 Filter rows by a column

QUESTION

Select the rows where the second column is greater than 75.

GIVEN

```
arr = np.array([[78, 85, 90],
                [45, 55, 60],
                [92, 88, 95],
                [65, 70, 72]])
```

ANSWER

PYTHON

```
print(arr[arr[:, 1] > 75])
```

OUTPUT

```
[[78 85 90]
 [92 88 95]]
```

ALTERNATIVE

```
print(arr[np.where(arr[:, 1] > 75)[0]])
```

`arr[:, 1]` is the whole second column, `> 75` makes a mask, and `arr[mask]` keeps the matching rows.

Q47 Consecutive differences

QUESTION

Calculate each day's temperature change from the previous day.

GIVEN

```
temps = np.array([25, 28, 26, 30, 32, 29])
```

ANSWER

PYTHON

```
print(np.diff(temps))
```

OUTPUT

```
[ 3 -2  4  2 -3]
```

ALTERNATIVE

```
print(temps[1:] - temps[:-1])
```

The result has one fewer element than the input. Positive means a rise, negative a fall.

Q48 Vectorize a formula

QUESTION

Calculate y = 3x^2 + 2x + 5 for x = 1 to 100 without a loop.

GIVEN

```
x = np.arange(1, 101)
```

ANSWER

PYTHON

```
y = 3 * x**2 + 2 * x + 5
print(y[:5])
print(y.shape)
```

OUTPUT

```
[10 21 38 61 90]
(100,)
```

ALTERNATIVE

```
y = 3 * np.square(x) + 2 * x + 5
```

NumPy applies the formula to all 100 values at once. For x = 1: 3 + 2 + 5 = 10. For x = 2: 12 + 4 + 5 = 21. Only the first five values are printed here.

Level 5 - Real-World Challenges

Questions 49-55 | Data analysis scenarios

Q49 Student marks analysis

QUESTION

A 2D array stores marks for five students across four subjects.

1. Calculate each student's average.
2. Find the highest-scoring student by average.
3. Calculate the average mark for each subject.
4. Count students whose average is at least 60.
5. Identify the highest individual mark and its position.

GIVEN

```
marks = np.array([
    [78, 85, 90, 66],
    [45, 55, 60, 50],
    [92, 88, 95, 91],
    [65, 70, 72, 68],
    [35, 40, 30, 45]
])
```

ANSWER

PYTHON

```
avg = marks.mean(axis=1)
print('Student averages :', avg)
print('Top student (idx):', np.argmax(avg))
print('Subject averages :', marks.mean(axis=0))
print('Averages >= 60   :', np.count_nonzero(avg >= 60))
print('Highest mark      :', marks.max())
print('Its position     :', tuple(int(i) for i in np.unravel_index(marks.argmax(),
marks.shape)))
```

OUTPUT

```
Student averages : [79.75 52.5  91.5  68.75 37.5 ]
Top student (idx): 2
Subject averages : [63.  67.6 69.4 64. ]
Averages >= 60   : 3
Highest mark      : 95
Its position     : (2, 2)
```

`axis=1` = one result per student (row); `axis=0` = one result per subject (column). Indexes start at 0, so student index 2 is the third student. `np.unravel_index` turns the flat `argmax` into (row, column).

Q50 Sales analysis

QUESTION

A shop records sales across three products (rows) over seven days (columns).

1. Calculate total sales for each product.
2. Calculate total sales per day.
3. Find the best-selling product.
4. Identify days when total sales exceeded 400.
5. Calculate the average daily sales for the shop.

GIVEN

```
sales = np.array([
    [120, 150, 100, 180, 200, 170, 220],
    [ 80,  90, 110,  95, 130, 125, 140],
    [200, 180, 210, 230, 190, 240, 260]
])
```

ANSWER

PYTHON

```
product_total = sales.sum(axis=1)
day_total = sales.sum(axis=0)
print('Total per product:', product_total)
print('Total per day    :', day_total)
print('Best product idx :', np.argmax(product_total))
print('Days over 400    :', np.where(day_total > 400)[0])
print('Avg daily sales  :', day_total.mean())
```

OUTPUT

```
Total per product: [1140  770 1510]
Total per day    : [400 420 420 505 520 535 620]
Best product idx : 2
Days over 400    : [1 2 3 4 5 6]
Avg daily sales  : 488.57142857142856
```

Rows are products, so a per-product total sums across columns (`axis=1`). A per-day total sums down the rows (`axis=0`). Day indexes are 0-based: index 2 is day 3.

Q51 Temperature analysis

QUESTION

Analyse ten days of temperature readings.

1. Find the mean, median, minimum and maximum.
2. Count days above 35 degrees.
3. Find the hottest and coldest day's indices.
4. Calculate the temperature changes between consecutive days.
5. Replace readings above 37 with the array's median.

GIVEN

```
temp = np.array([28, 31, 29, 35, 38, 32, 27, 40, 36, 30])
```

ANSWER

PYTHON

```python
print('Mean/median/min/max:', temp.mean(), np.median(temp), temp.min(), temp.max())
print('Days above 35     :', np.count_nonzero(temp > 35))
print('Hottest/coldest idx:', temp.argmax(), temp.argmin())
print('Daily changes      :', np.diff(temp))
print('Capped at median   :', np.where(temp > 37, np.median(temp), temp))
```

OUTPUT

```
Mean/median/min/max: 32.6 31.5 27 40
Days above 35     : 3
Hottest/coldest idx: 7 6
Daily changes      : [ 3 -2  6  3 -6 -5 13 -4 -6]
Capped at median   : [28.  31.  29.  35.  31.5 32.  27.  31.5 36.  30. ]
```

The temperatures above 35 are 38, 40 and 36, so the count is 3. `np.count_nonzero(temp > 35)` counts matches; `np.sum(np.where(...))` would add up the **indices** instead.

Q52 Data cleaning

QUESTION

In `data`, the value -999 represents a missing observation.

1. Identify the missing-value positions.
2. Replace missing values with the mean of valid observations.
3. Calculate the cleaned array's mean.
4. Return an array containing only valid observations.

GIVEN

```python
data = np.array([10, 20, -999, 30, 40, -999, 50])
```

ANSWER

PYTHON

```python
missing = data == -999
print('Missing positions:', np.where(missing)[0])
valid = data[~missing]
print('Valid values     :', valid)
cleaned = np.where(missing, valid.mean(), data)
print('Cleaned array    :', cleaned)
print('Cleaned mean     :', cleaned.mean())
```

OUTPUT

```
Missing positions: [2 5]
Valid values     : [10 20 30 40 50]
Cleaned array    : [10. 20. 30. 30. 40. 30. 50.]
Cleaned mean     : 30.0
```

The mean must be computed from the **valid** values only, otherwise -999 drags it down. `~` flips a Boolean mask.

Q53 Image matrix operations

QUESTION

Treat a 5 x 5 array as a grayscale image with pixel values 0 to 255.

1. Increase brightness by adding 30 while keeping values between 0 and 255.
2. Convert every pixel below 100 to zero.
3. Extract the central 3 x 3 region.
4. Flip the image horizontally and vertically.

GIVEN

```
img = np.array([
    [ 10,  60, 110, 160, 210],
    [ 20,  70, 120, 170, 220],
    [ 30,  80, 130, 180, 230],
    [ 40,  90, 140, 190, 240],
    [ 50, 100, 150, 200, 250]
])
```

ANSWER

PYTHON

```
bright = np.clip(img + 30, 0, 255)
print(bright)
print(np.where(img < 100, 0, img))
print(img[1:4, 1:4])
print(img[:, ::-1])   # horizontal flip
print(img[::-1, :])   # vertical flip
```

OUTPUT

```
[[ 40  90 140 190 240]
 [ 50 100 150 200 250]
 [ 60 110 160 210 255]
 [ 70 120 170 220 255]
 [ 80 130 180 230 255]]
[[  0   0 110 160 210]
 [  0   0 120 170 220]
 [  0   0 130 180 230]
 [  0   0 140 190 240]
 [  0 100 150 200 250]]
[[ 70 120 170]
 [ 80 130 180]
 [ 90 140 190]]
[[210 160 110  60  10]
 [220 170 120  70  20]
 [230 180 130  80  30]
 [240 190 140  90  40]
 [250 200 150 100  50]]
[[ 50 100 150 200 250]
 [ 40  90 140 190 240]
 [ 30  80 130 180 230]
 [ 20  70 120 170 220]
 [ 10  60 110 160 210]]
```

ALTERNATIVE

```
print(np.fliplr(img))   # horizontal flip
print(np.flipud(img))   # vertical flip
```

`np.clip(a, 0, 255)` enforces both limits. If an image is stored as `uint8`, adding 30 can silently wrap around (250 + 30 = 24), so clip in a wider integer type first.

Q54 Financial returns

QUESTION

Analyse percentage returns of a price series.

1. Calculate percentage returns between consecutive prices.
2. Find the average return.
3. Find the largest positive and negative returns.
4. Calculate cumulative returns relative to the first price.

GIVEN

```
prices = np.array([100, 105, 102, 110, 108, 115])
```

ANSWER

PYTHON

```
returns = (prices[1:] - prices[:-1]) / prices[:-1] * 100
print('Returns %   :', np.round(returns, 2))
print('Average     :', round(returns.mean(), 2))
print('Best / worst:', round(returns.max(), 2), round(returns.min(), 2))
cum = (prices / prices[0] - 1) * 100
print('Cumulative %:', np.round(cum, 2))
```

OUTPUT

```
Returns %   : [ 5.   -2.86  7.84 -1.82  6.48]
Average     : 2.93
Best / worst: 7.84 -2.86
Cumulative %: [ 0.  5.  2. 10.  8. 15.]
```

ALTERNATIVE

```
returns = np.diff(prices) / prices[:-1] * 100
```

`prices[1:]` are the current prices and `prices[:-1]` the previous ones, so all returns are computed together. Return = (current - previous) / previous x 100.

Q55 Challenge: NumPy without loops

QUESTION

Solve each task with no explicit `for` or `while` loop.

1. Find all values greater than 40.
2. Calculate each row's mean and standard deviation.
3. Normalize each column independently.
4. Replace each value below its row's mean with zero.
5. Sort each row independently.
6. Return the rows in descending order of their row means.

GIVEN

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90],
    [15, 25, 35]
])
```

ANSWER

PYTHON

```
print(arr[arr > 40])                                    # 1
row_mean = arr.mean(axis=1)
print(row_mean, arr.std(axis=1))                        # 2
print((arr - arr.min(axis=0)) / (arr.max(axis=0) - arr.min(axis=0)))   # 3
print(np.where(arr < arr.mean(axis=1, keepdims=True), 0, arr))         # 4
print(np.sort(arr, axis=1))                             # 5
print(arr[np.argsort(row_mean)[::-1]])                  # 6
```

OUTPUT

```
[50 60 70 80 90]
[20. 50. 80. 25.] [8.16496581 8.16496581 8.16496581 8.16496581]
[[0.         0.         0.        ]
 [0.5        0.5        0.5       ]
 [1.         1.         1.        ]
 [0.08333333 0.08333333 0.08333333]]
[[ 0 20 30]
 [ 0 50 60]
 [ 0 80 90]
 [ 0 25 35]]
[[10 20 30]
 [40 50 60]
 [70 80 90]
 [15 25 35]]
[[70 80 90]
 [40 50 60]
 [15 25 35]
 [10 20 30]]
```

Task 3: subtracting a per-column minimum broadcasts down the rows. Task 4: `keepdims=True` keeps the row means as shape (4, 1) so they broadcast across each row. Task 6: `argsort` gives the order, `[::-1]` reverses it, and indexing the rows with it reorders them.

Day 1 - Slicing Drills

Extra practice | Indexing and slicing with steps

D1 Access by index

QUESTION

Print the second, fifth and second-last elements of the array.

GIVEN

```
arr = np.array([15, 25, 35, 45, 55, 65, 75, 85])
```

ANSWER

PYTHON

```
print(arr[1], arr[4], arr[-2])
```

OUTPUT

```
25 55 75
```

Index 1 is the second element, index 4 the fifth, and `-2` the second-last.

D2 Every second element

QUESTION

Print every second element, starting from the first and ending at the ninth element (90).

GIVEN

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
```

ANSWER

PYTHON

```
print(arr[0:9:2])
```

OUTPUT

```
[10 30 50 70 90]
```

ALTERNATIVE

```
print(arr[:9:2])
```

Slice syntax uses colons: `arr[start:stop:step]`. A form like `arr[1, 90, 2]` is not a slice.

D3 Reverse, excluding the ends

QUESTION

Print the array in reverse order, excluding the first and last elements.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
```

ANSWER

PYTHON

```
print(arr[-2:0:-1])
```

OUTPUT

```
[90 80 70 60 50 40 30 20]
```

ALTERNATIVE

```
print(arr[1:-1][::-1])
```

With a negative step, the start must be to the **right** of the stop. `arr[1:-1:-1]` is empty because it starts left of where it stops.

D4 Last four, reversed

QUESTION

Print the last four elements in reverse order.

GIVEN

```
arr = np.array([12, 24, 36, 48, 60, 72, 84, 96])
```

ANSWER

PYTHON

```
print(arr[-1:-5:-1])
```

OUTPUT

```
[96 84 72 60]
```

ALTERNATIVE

```
print(arr[-4:][::-1])
```

Start at `-1` (the last element), go backwards, and stop before `-5`.

D5 Elements at even index positions

QUESTION

Print the elements at index positions 0, 2, 4, 6 and 8.

GIVEN

```
arr = np.array([5, 12, 7, 20, 9, 14, 3, 18, 11, 6])
```

ANSWER

PYTHON

```
print(arr[::2])
```

OUTPUT

```
[ 5  7  9  3 11]
```

ALTERNATIVE

```
print(arr[arr % 2 == 0])   # selects even VALUES, a different task
```

`arr[::2]` selects by **position**. `arr[arr % 2 == 0]` selects by **value**. Know which one the question wants.

D6 Reverse slice of a range

QUESTION

Print the elements from index 2 through index 7, in reverse order.

GIVEN

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
```

ANSWER

PYTHON

```
print(arr[7:1:-1])
```

OUTPUT

```
[80 70 60 50 40 30]
```

ALTERNATIVE

```
print(arr[2:8][::-1])
```

The stop index is excluded, so `arr[7:1:-1]` runs from index 7 down to index 2.

Common Mistakes to Remember

Patterns that came up repeatedly while practising

Trap	Remember
`np.where(cond)` returns indices	Counting matches: use `np.count_nonzero(cond)` or `np.sum(cond)`
`np.max` vs `np.argmax`	`max` gives the value, `argmax` gives its index
`arr[:-1]` vs `arr[::-1]`	`[:-1]` drops the last element; `[::-1]` reverses
`flatten()` is a method	Write `arr.flatten()`, not `np.flatten(arr)`
`np.array()` needs one list	`np.array([10, 20, 30])`, not `np.array(10, 20, 30)`
`sum` vs `mean` with `axis`	Right axis, wrong function is a common slip; re-read the question
`axis=0` vs `axis=1`	`axis=0` gives one result per column; `axis=1` one result per row
`reshape` element count	rows x columns must equal the array size
`arange` vs `linspace`	`arange` excludes the stop; `linspace` includes the endpoint
Backward slices	Negative step: start must be to the right of stop, e.g. `arr[7:1:-1]`
Position vs value	`arr[::2]` selects by position; `arr[arr % 2 == 0]` by value
Zero-based indexing	The third element is `arr[2]`
`keepdims=True`	Keeps row statistics as a column so they broadcast across each row
`&`, `|`, `~` with brackets	`arr[(arr > 10) & (arr < 50)]`, not `and` / `or`